



PIC-UPV
PERTE CHIP CHAIR

Lab Book 4.0 Circuits and Compact Models

Student: Antonio González López y María Ros Téllez

Instructions

General

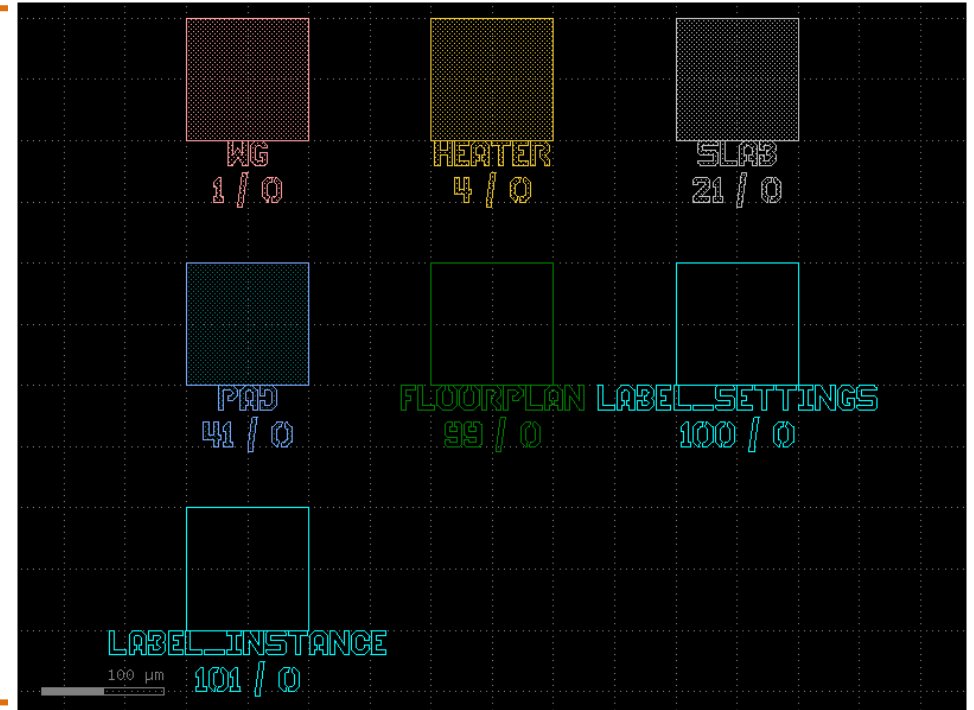
SCRIPTS

- From GitHub →
 - Fork the repo: <https://github.com/pascualmunoz/cifoin-lab4.git>
 - Clone the repo using VSCode
 - Open folder / repo in VSCode
 - Open terminal: uv sync
- Run the Jupyter Notebook:
 - LAYOUT_STUDENT.ipynb
- **Points**
 - **LO1: 3 points**
 - **LO2: 3 points**
 - **LO3: 1 points**
 - **LO4: 3 points**

Learning outcome #1a – Notebook section 1. Technology

“We will work with the demo UPVfab PDK. Let's view the layer distribution of this PDK”

```
LAYER_VIEWS = PDK.layer_views  
c = LAYER_VIEWS.preview_layerset()  
c.show()  
c.plot()
```



Una PDK es el conjunto de capas, reglas y componentes que la foundry proporciona para diseñar circuitos fabricables con su proceso. En general, las capas representan los distintos materiales o pasos de fabricación, cada uno identificado con un número (GDS) y una etiqueta. Para nuestra PDK encontramos las siguientes capas: WG (1/0), la guía de onda o núcleo óptico; HEATER (4/0), los calentadores para sintonización térmica; SLAB (21/0), la losa o región parcialmente grabada; PAD (41/0), los contactos metálicos; FLOORPLAN (99/0), que delimita el área del chip; y LABEL_SETTINGS (100/0) y LABEL_INSTANCE (101/0), las capas auxiliares de etiquetado.

Learning outcome #1b – Notebook section 1. Technology – DC

“Use your design results from Lab2 and Lab3 to create layout instances of your designed components: DCs, MMI, MZIs & Ring Resonators”

```
# DC Lpi=93.83

PDK.activate()

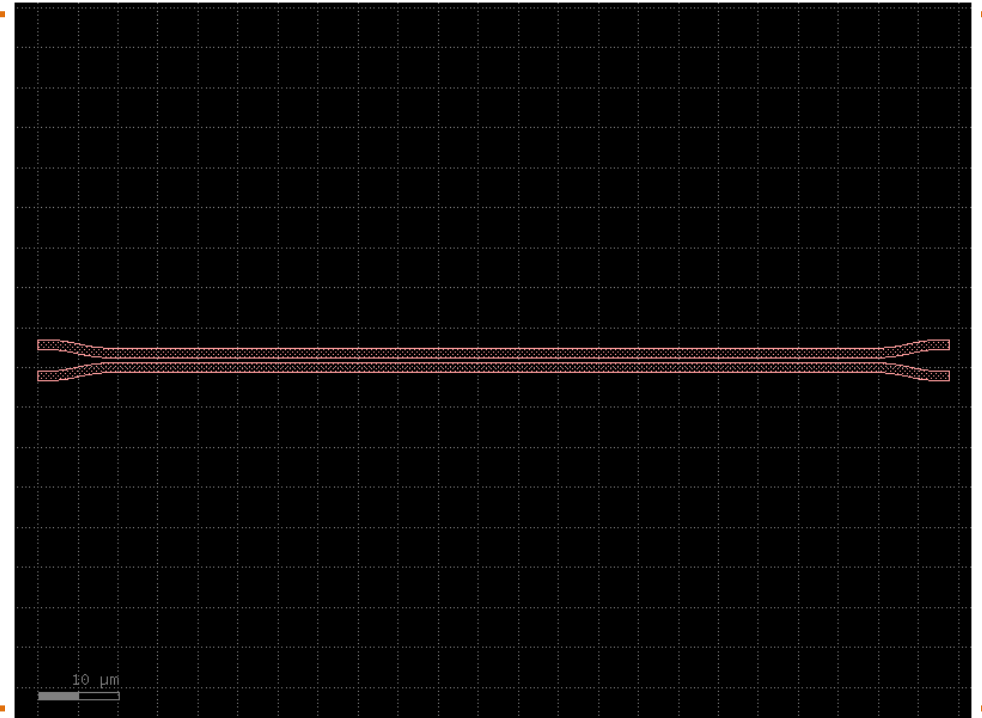
xs_wg = gf.cross_section.strip(width=1.2, layer='WG')

# Creamos nuestro componente principal
c = gf.Component("my_custom_dc")

# 1. Mantra: Add references
# Instanciamos el acoplador direccional
dc_ref = c.add_ref(
    gf.components.coupler(
        length=93.83,
        gap=0.6,
        cross_section=xs_wg
    )
)

# Añadimos los puertos al componente exterior
c.add_ports(dc_ref.ports)

# Visualización
c.plot()
c.show()
```



Este es el DC (acoplador direccional) que diseñamos en el Lab2: un acoplador 2x2 basado en dos guías acopladas por proximidad, con constante de acoplamiento objetivo $K = 0.5$ (divisor 50/50). La longitud de acoplo se ha fijado en $L_{pi} = 93.83 \mu\text{m}$, valor de la longitud necesaria para transferir el 50 % de la potencia para ese gap y sección. Las dimensiones relevantes son: gap = $0.6 \mu\text{m}$, ancho de guía $W = 1.2 \mu\text{m}$ y longitud de acoplo $L = 93.83 \mu\text{m}$.

Learning outcome #1b – Notebook section 1. Technology – MMI

“Use your design results from Lab2 and Lab3 to create layout instances of your designed components: DCs, MMIs, MZIs & Ring Resonators”

```
# MMI 2x2 L_mmi=34.518 W_mmi=6.6
PDK.activate()

# Creamos el componente global de nuestro MMI
c = gf.Component("custom_mmi2x2_lab")

# Definimos las secciones transversales extraídas del Lab 2
xs_multimodo = gf.cross_section.strip(width=6.6, layer='WG')
xs_acceso = gf.cross_section.strip(width=1.0, layer='WG')

# --- MANTRA: 1. Add references ---
body_ref = c.add_ref(gf.components.straight(length=34.518, cross_section=xs_multimodo))

# Generamos el componente base para las guías de acceso
wg_acceso = gf.components.straight(length=10.0, cross_section=xs_acceso)

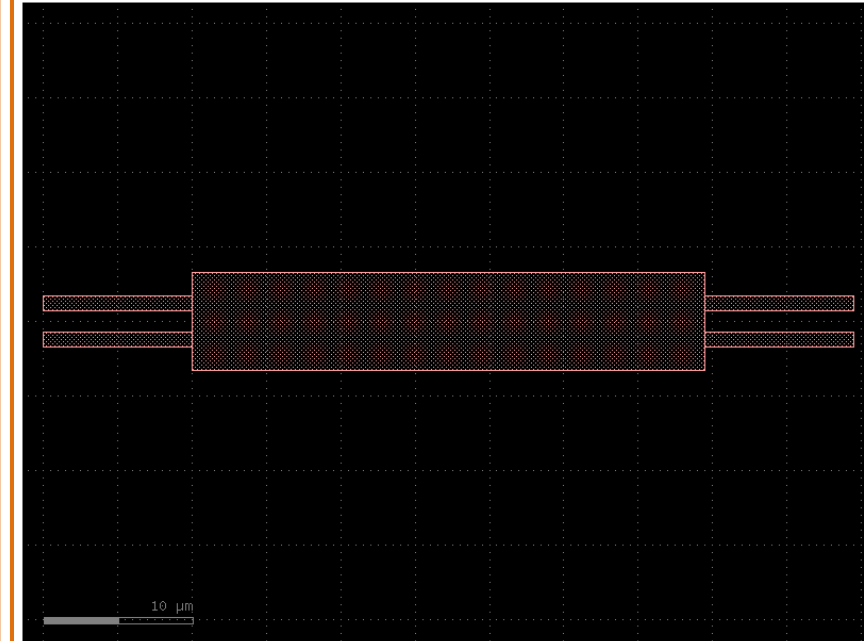
# Instanciamos los 4 accesos
in_top = c.add_ref(wg_acceso)
in_bot = c.add_ref(wg_acceso)
out_top = c.add_ref(wg_acceso)
out_bot = c.add_ref(wg_acceso)

# --- MANTRA: 2. Move them ---
# Colocamos las entradas en la parte izquierda
in_top.dmovex(-10.0).dmovey(1.2)
in_bot.dmovex(-10.0).dmovey(-1.2)

# Colocamos las salidas en la parte derecha (al final del cuerpo del MMI)
out_top.dmovex(34.518).dmovey(1.2)
out_bot.dmovex(34.518).dmovey(-1.2)

# --- MANTRA: 3. Puertos
c.add_port(name="o1", port=in_top.ports["o1"])
c.add_port(name="o2", port=in_bot.ports["o1"])
c.add_port(name="o3", port=out_top.ports["o2"])
c.add_port(name="o4", port=out_bot.ports["o2"])

# Visualización
c.plot()
c.show()
```



Este es el MMI 2x2 que diseñamos en el Lab2, basado en interferencia por imágenes pareadas (paired interference), con constante de acoplo objetivo $K = 0.5$ (divisor 50/50). Las dimensiones relevantes son $W_{\text{MMI}} = 6.6 \mu\text{m}$ y $L_{\text{MMI}} = 34.518 \mu\text{m}$ (ancho y longitud de la región multimodo), y las posiciones de las entradas/salidas respecto al centro están a $\pm dY = 1.2 \mu\text{m}$. Las guías de acceso tienen un ancho de $1.0 \mu\text{m}$ y una longitud de $10 \mu\text{m}$, conectadas al cuerpo multimodo (ancho $6.6 \mu\text{m}$) por los cuatro puertos o1, o2 (entradas) y o3, o4 (salidas).

Learning outcome #1b – Notebook section 1. Technology – MZI

“Use your design results from Lab2 and Lab3 to create layout instances of your designed components: DCs, MMI, MZIs & Ring Resonators”

```
# MZI
PDK.activate()

# Creamos nuestro componente contenedor
c = gf.Component("mzi_balanceado_lab")

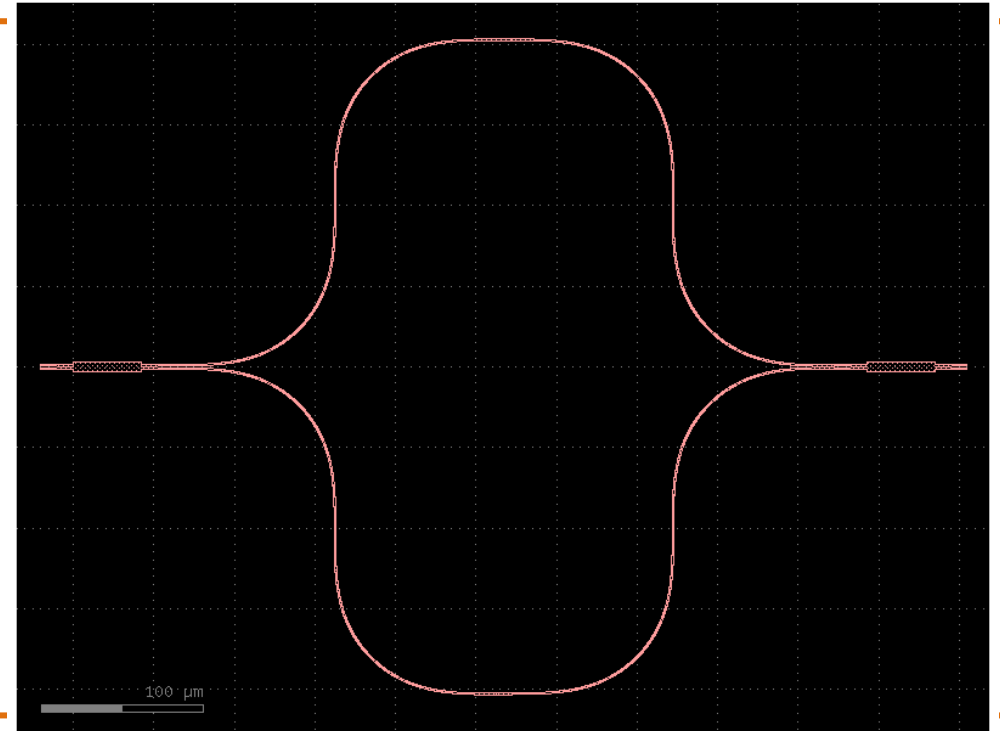
xs_ancha = gf.cross_section.strip(width=1.0)

# --- 1. Mantra: Add references ---
mzi_base = gf.components.mzi2x2_2x2(
    delta_length=0.0,
    cross_section=xs_ancha
)

# Añadimos la referencia a nuestro diseño
mzi_inst = c.add_ref(mzi_base)

# --- 2. Mantra: Connect them (exponemos los puertos) ---
c.add_ports(mzi_inst.ports)

# Visualizamos en el notebook y abrimos en KLayout
c.plot()
c.show()
```



Este es el MZI 2x2 diseñado a partir de dos acopladores 2x2, usando el componente nativo mzi2x2_2x2. Se trata de un MZI balanceado, con $\text{delta_length} = 0 \mu\text{m}$, por lo que ambos brazos tienen la misma longitud. El componente expone los puertos del MZI original (add_ports) y queda preparado para usarse como bloque dentro del layout.

Learning outcome #1b – Notebook section 1. Technology – **RING RESONATOR**

“Use your design results from Lab2 and Lab3 to create layout instances of your designed components: DCs, MMI, MZIs & Ring Resonators”

```
# RING RESONATOR    perimeter=75um --> ~9.9um
PDK.activate()

# Creamos el contenedor principal
c = gf.Component("anillo_simple_lab")

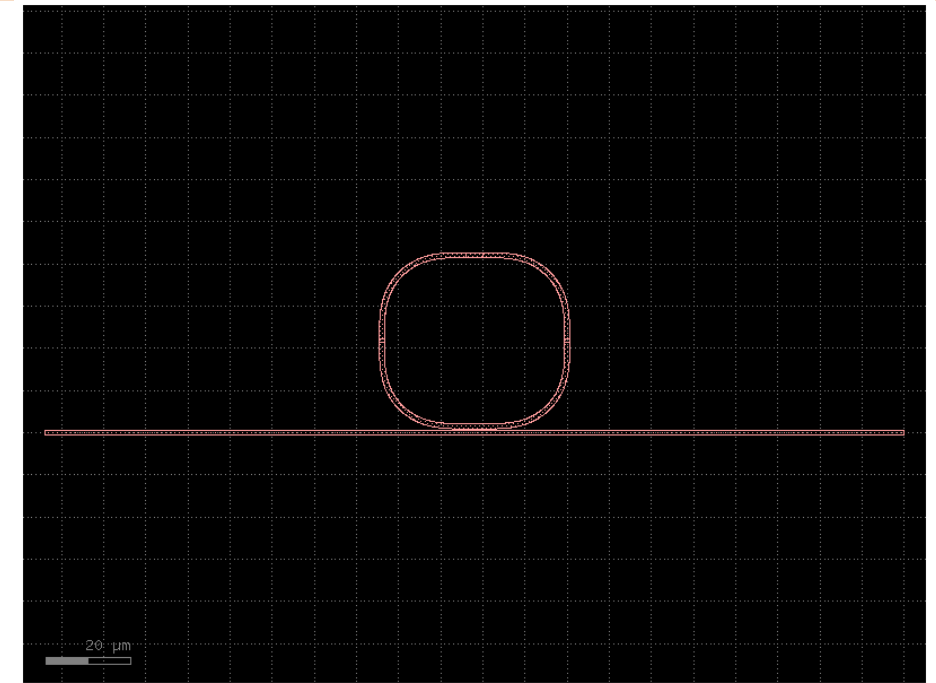
# Definimos la sección transversal con los requisitos de la práctica
xs_anillo = gf.cross_section.strip(width=1.2, layer='WG', radius_min=10.0)

# --- 1. Mantra: Add references ---
# Generamos el componente base
ring_base = gf.components.ring_single(
    radius=20.0,
    gap=0.3,
    cross_section=xs_anillo
)

# Lo instanciamos en nuestro componente
anillo_ref = c.add_ref(ring_base)

# --- 2. Mantra: Connect them (exponemos los puertos) ---
c.add_ports(anillo_ref.ports)

# Visualización en notebook y en KLayout
c.plot()
c.show()
```



Este es el resonador en anillo (ring resonator) tipo all-pass que diseñamos en el Lab3: un anillo acoplado por proximidad a una guía recta (bus), basado en ring_single. Las dimensiones relevantes son radio = 9.9 µm, gap = 0.3 µm (separación entre el anillo y la guía de acceso) y ancho de guía W = 1.2 µm. El perímetro del anillo es de aproximadamente $2 \cdot \pi \cdot 9.9 \approx 75$ µm. El componente expone los puertos de entrada y salida de la guía bus (add_ports) para poder conectarlo en el chip final.

Learning outcome #2a – Notebook section 2. Layout Fundamentals

“Once that we have a working 'new' component, we shall convert it into a Cell. This will allow us to have a hierarchical design”

```
wgw = 0.5
dieL = 5000
dieW = 5000
border = 125
layer_wg = "WG"
layer_box = "FLOORPLAN"

# Die specifications (Chip)
box = gf.Component()

obox = box.add_ref(gf.components.rectangle(size=(dieW,dieL),layer=layer_box))
ibox = box.add_ref(gf.components.rectangle(size=(dieW-border*2,dieL-border*2),layer=layer_box)).dmovex(border).dmovey(border)
box = gf.boolean(A=obox, B=ibox, operation="A-B", layer="FLOORPLAN")

# Adding ports to a component
box.add_port(name="block@org", center=[border,border], width=1, orientation=0, layer=layer_box)

box.draw_ports()
box.show()
box.plot()
```



Es el marco del die (floorplan): un rectángulo hueco que define el área del chip. Se crea con una operación booleana A-B de dos rectángulos en la capa FLOORPLAN, dando un marco de 125 µm de borde sobre un die de 5000 × 5000 µm. Incluye un puerto de referencia (block@org) en la esquina para anclar los demás componentes.

Learning outcome #2a – Notebook section 2. Layout Fundamentals – DC

“Use your design results from Lab2 and Lab3 to create layout instances of your designed components: DCs, MMI, MZIs & Ring Resonators” → *Make cells for all your components*

```
PDK.activate()

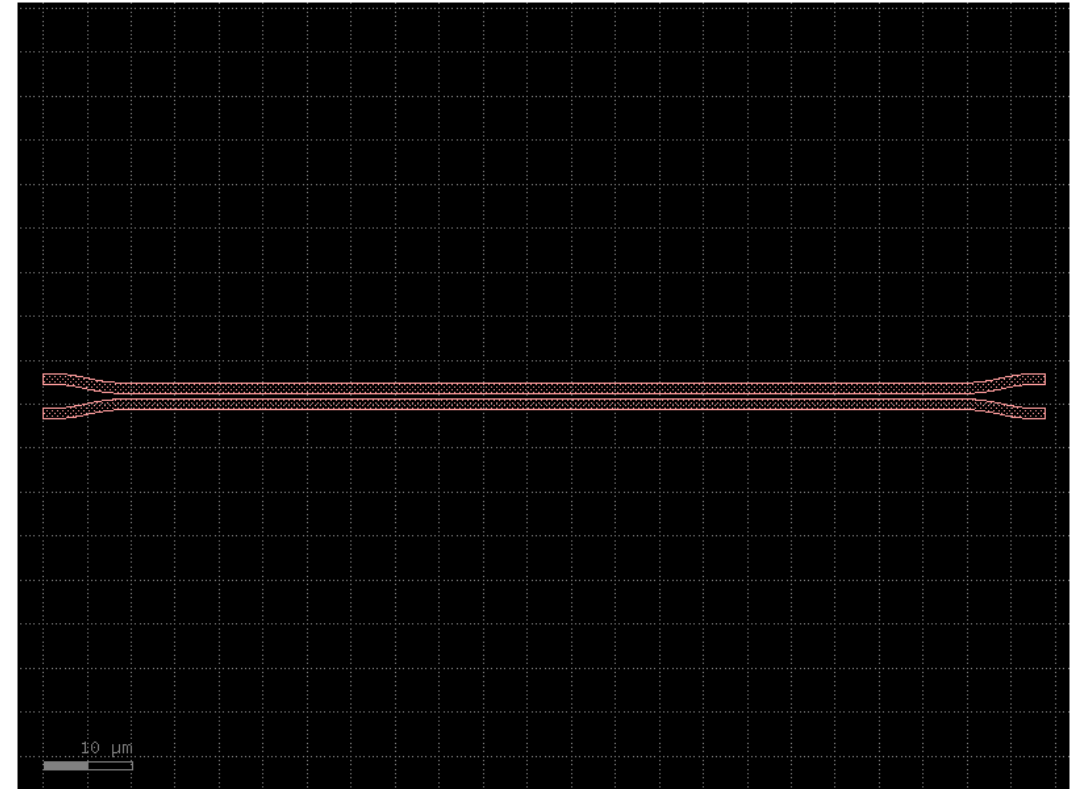
@gf.cell
def dc_lab(length: float = 93.83, gap: float = 0.6):
    c = gf.Component()
    xs_wg = gf.cross_section.strip(width=1.2, layer='WG')

    # 1. Mantra: Add references
    dc_ref = c.add_ref(
        gf.components.coupler(
            length=length,
            gap=gap,
            cross_section=xs_wg,
        )
    )

    # Añadimos los puertos al componente exterior
    c.add_ports(dc_ref.ports)

    return c

# Lo instanciamos y visualizamos
mi_dc = dc_lab(length=93.83, gap=0.6)
mi_dc.plot()
mi_dc.show()
```



Learning outcome #2a – Notebook section 2. Layout Fundamentals – MMI

“Use your design results from Lab2 and Lab3 to create layout instances of your designed components: DCs, MMIs, MZIs & Ring Resonators” → *Make cells for all your components*

```
PDK.activate()

@gf.cell
def mmi2x2_lab(L_mmi: float = 34.518, W_mmi: float = 6.6, L_acceso: float = 10.0, dY: float = 1.2):
    c = gf.Component()

    # Secciones transversales extraídas del Lab 2
    xs_multimodo = gf.cross_section.strip(width=W_mmi, layer='WG')
    xs_acceso = gf.cross_section.strip(width=1.0, layer='WG')

    # --- MANTRA: 1. Add references ---
    body_ref = c.add_ref(gf.components.straight(length=L_mmi, cross_section=xs_multimodo))

    # Componente base para las guías de acceso
    wg_acceso = gf.components.straight(length=L_acceso, cross_section=xs_acceso)

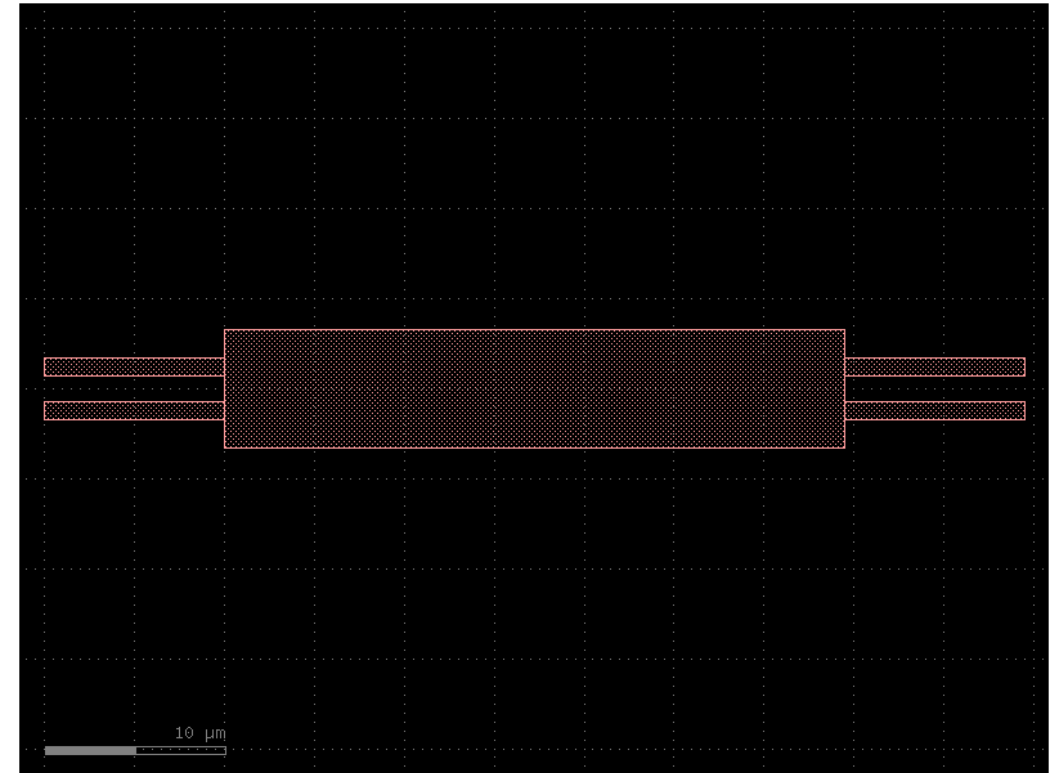
    in_top = c.add_ref(wg_acceso)
    in_bot = c.add_ref(wg_acceso)
    out_top = c.add_ref(wg_acceso)
    out_bot = c.add_ref(wg_acceso)

    # --- MANTRA: 2. Move them ---
    in_top.dmovex(-L_acceso).dmovex(dY)
    in_bot.dmovex(-L_acceso).dmovex(-dY)
    out_top.dmovex(L_mmi).dmovex(dY)
    out_bot.dmovex(L_mmi).dmovex(-dY)

    # --- MANTRA: 3. Puentes ---
    c.add_port(name="o1", port=in_top.ports["o1"])
    c.add_port(name="o2", port=in_bot.ports["o1"])
    c.add_port(name="o3", port=out_top.ports["o2"])
    c.add_port(name="o4", port=out_bot.ports["o2"])

    return c

# Lo instanciamos y visualizamos
mi_mmi = mmi2x2_lab(L_mmi=34.518, W_mmi=6.6)
mi_mmi.plot()
mi_mmi.show()
```



Learning outcome #2a – Notebook section 2. Layout Fundamentals – MZI

“Use your design results from Lab2 and Lab3 to create layout instances of your designed components: DCs, MMI, MZIs & Ring Resonators” → *Make cells for all your components*

```
# MZI
PDK.activate()

@gf.cell
def mzi_balanced_2x2(delta_length=0.0):
    # Creamos nuestro componente contenedor
    c = gf.Component("mzi_balanceado_lab")

    xs_ancha = gf.cross_section.strip(width=1.0)

    # --- 1. Mantra: Add references ---
    mzi_base = gf.components.mzi2x2_2x2(
        delta_length=delta_length,
        cross_section=xs_ancha
    )

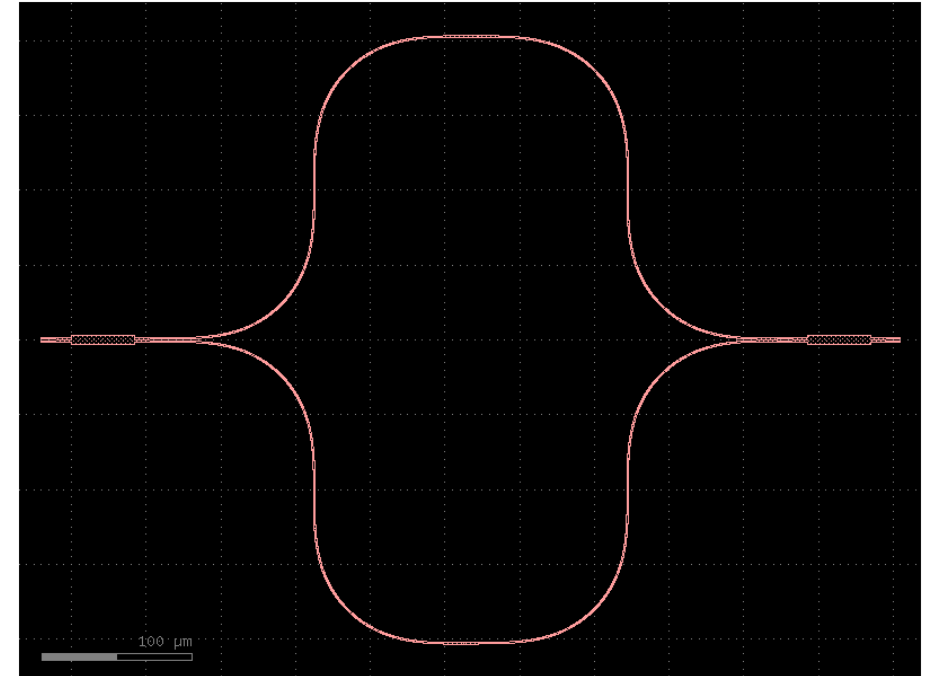
    # Añadimos la referencia a nuestro diseño
    mzi_inst = c.add_ref(mzi_base)

    # --- 2. Mantra: Connect them ---
    c.add_ports(mzi_inst.ports)

    return c

mi_mzi = mzi_balanced_2x2(delta_length=0.0)

# Visualizamos en el notebook y abrimos en KLayout
mi_mzi.plot()
mi_mzi.show()
```



Learning outcome #2a – Notebook section 2. Layout Fundamentals – MZI

“Use your design results from Lab2 and Lab3 to create layout instances of your designed components: DCs, MMI, MZIs & Ring Resonators” → *Make cells for all your components*

```
# RING RESONATOR    perimeter=75um --> ~9.9um
PDK.activate()

@gf.cell
def ring_resonator_lab(radius=9.9, gap=0.3):
    # Creamos el contenedor principal con un nombre único
    c = gf.Component("anillo_simple_lab")

    # Definimos la sección transversal con los requisitos de la práctica
    xs_anillo = gf.cross_section.strip(width=1.2, layer='WG', radius_min=10.0)

    # --- 1. Mantra: Add references ---
    # Generamos el componente base con nuestros parámetros paramétricos
    ring_base = gf.components.ring_single(
        radius=radius,
        gap=gap,
        cross_section=xs_anillo
    )

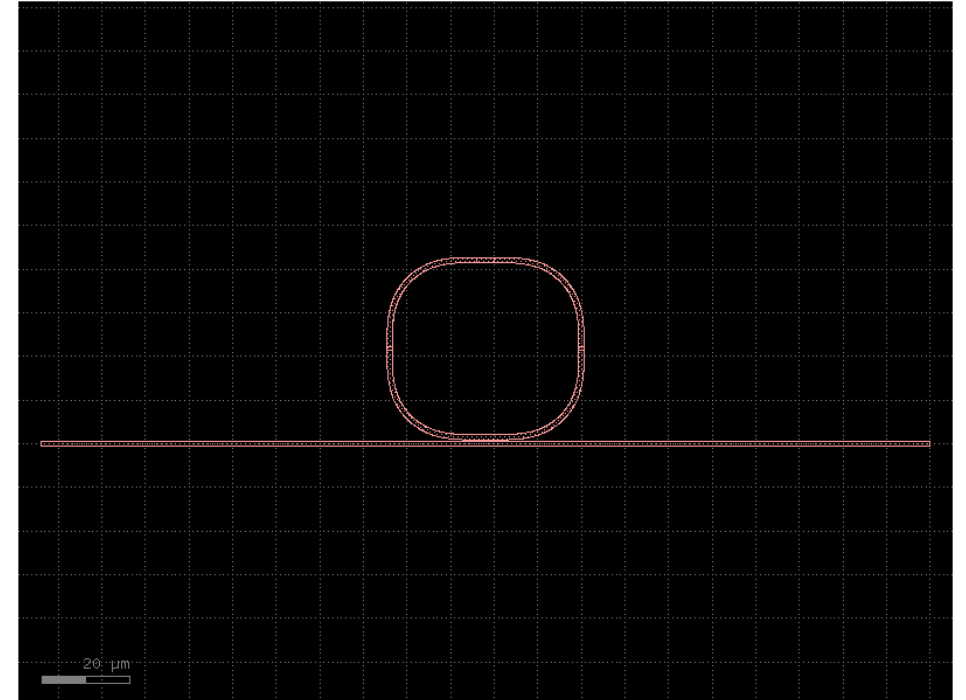
    # Lo instanciamos en nuestro componente
    anillo_ref = c.add_ref(ring_base)

    # --- 2. Mantra: Connect them ---
    # Como ya está montado, solo necesitamos exponer los puertos
    # hacia el exterior para poder conectarlo en el chip final
    c.add_ports(anillo_ref.ports)

    return c

# Probamos el componente con radio 20 um y gap 0.3 um
mi_anillo = ring_resonator_lab(radius=20.0, gap=0.3)

# Visualización en notebook y en KLayout
mi_anillo.plot()
mi_anillo.show()
```



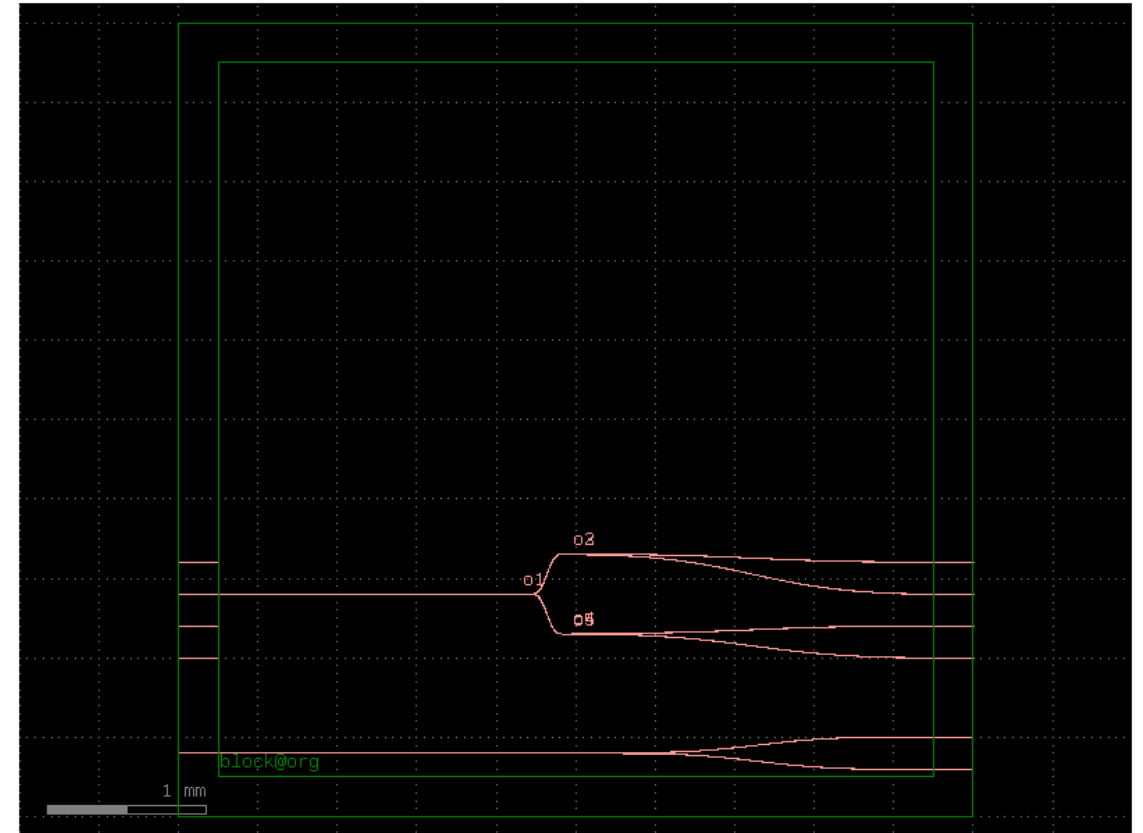
Learning outcome #3 – Notebook section 3. Complete layout

Python code block 1: Se crea el componente principal main y se añade el die (die(...) con dieW=5000 y capa FLOORPLAN) como referencia. Este die aporta el marco del chip y el puerto de anclaje block@org.

Python code block 2: Se instancia un MMI 1x2 de prueba (cells.mmi1x2()) y se coloca dentro del die, centrado en X (block@org.dx + 0.5·dieW) y desplazado 150 µm en Y respecto al punto de origen.

Python code block 3: Se añaden las guías de entrada/salida (straight de 500 µm) en los bordes del die. strin entra por la izquierda alineada con o1; strout1 y strout2 se rotan 180° y se colocan en el borde derecho (dmovex(dieW)), alineadas con las salidas o3 y o2 del MMI. Sirven para "cortar" en un tramo recto.

Python code block 4: Se enrutan las guías al MMI mediante curvas S (route_single_sbend): conecta strin → o1 (entrada), o2 → strout2 y o3 → strout1 (salidas). Finalmente, main.plot() y main.show() visualizan el layout completo.



Learning outcome #4 – Notebook section 4. Exercises – A1

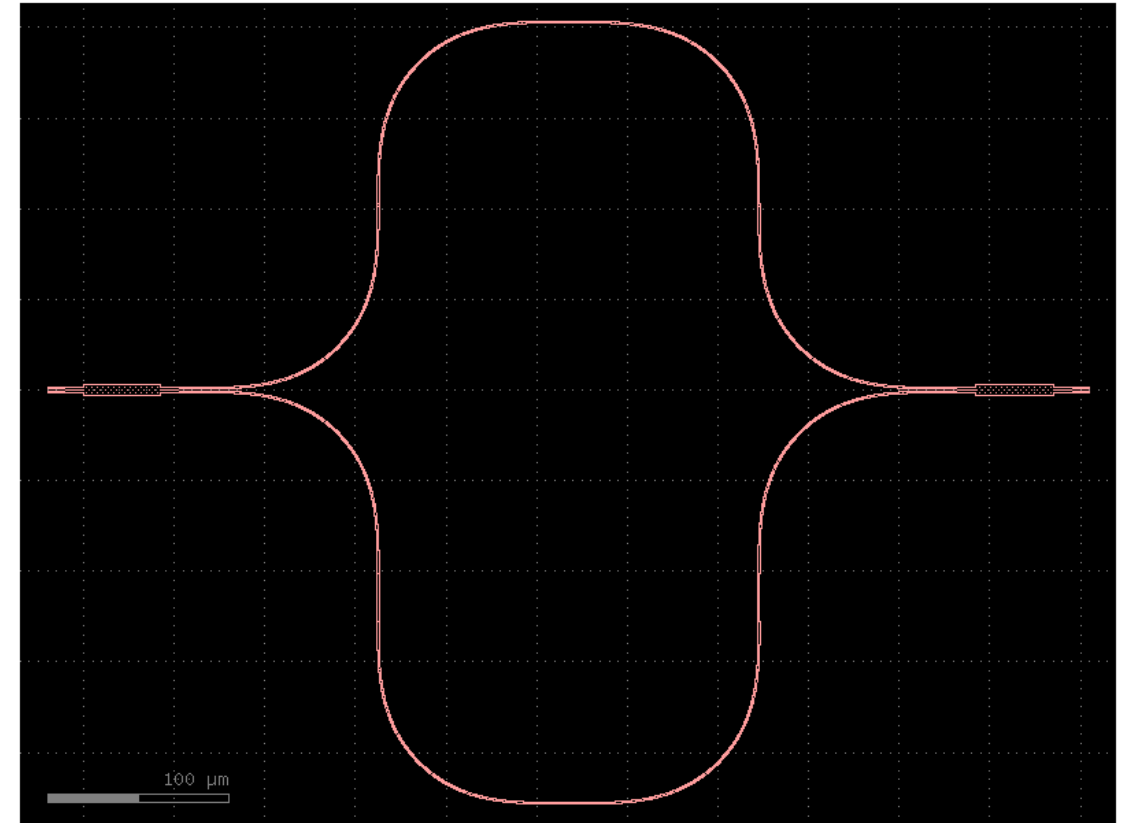
```
@gf.cell
def mzi_unbalanced_2x2(delta_length=0.0):
    c = gf.Component()

    # mzi2x2_2x2 es la función nativa específica para MZIs con MMIs 2x2
    _mzi = gf.components.mzi2x2_2x2(
        delta_length=delta_length,
        cross_section='strip'
    )

    mzi_ref = c.add_ref(_mzi)
    c.add_ports(mzi_ref.ports)

    return c

# Lo probamos
mi_mzi = mzi_unbalanced_2x2(delta_length=50)
mi_mzi.plot()
```



Learning outcome #4 – Notebook section 4. Exercises – A2

```
import gdsfactory as gf
from functools import partial

def heater_arm(**kwargs):
    kwargs.pop('cross_section', None)
    return gf.components.straight_heater_metal(**kwargs)

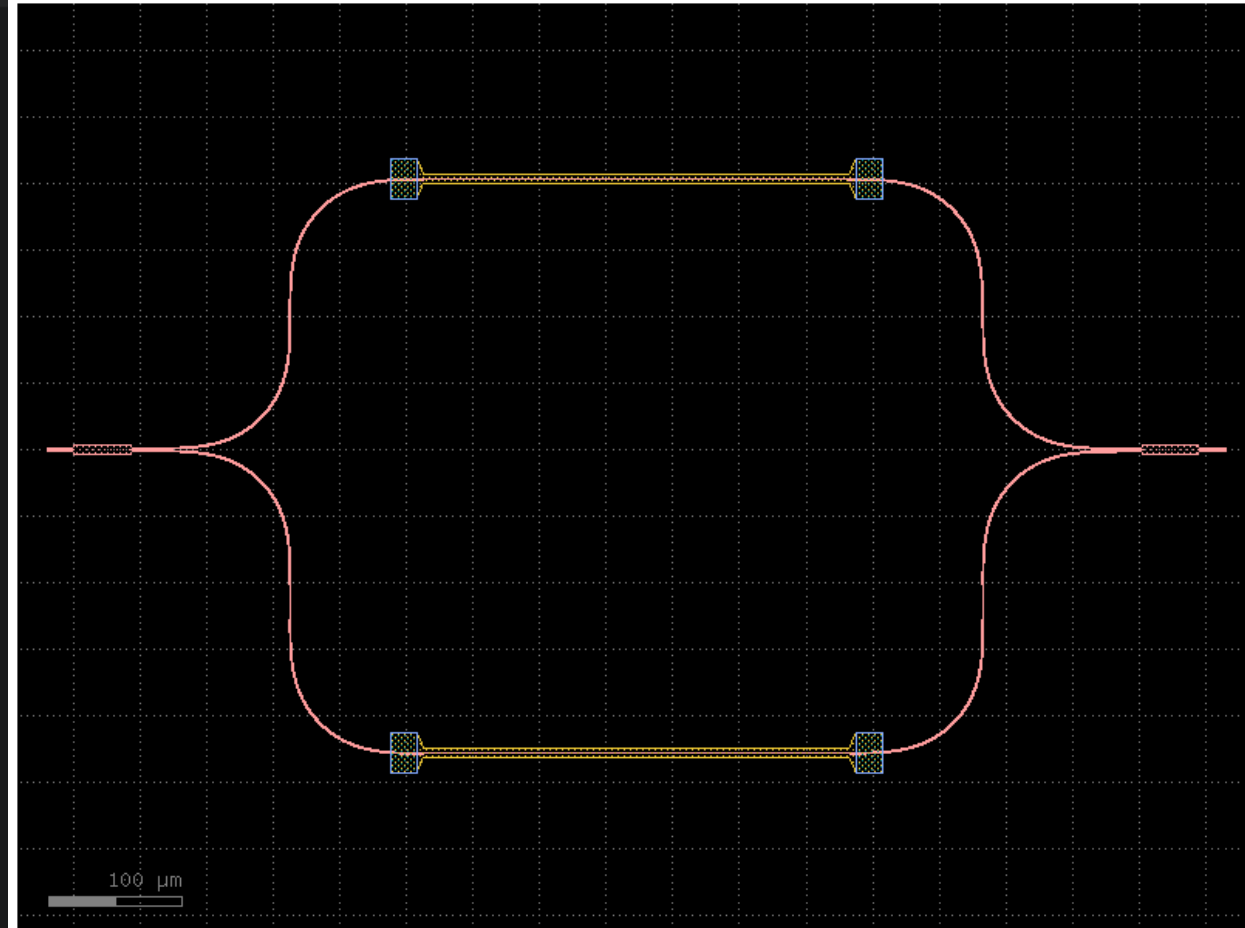
@gf.cell
def mzi_unbalanced_2x2_tunable(delta_length=0.0):
    c = gf.Component()

    _mzi = gf.components.mzi2x2_2x2(
        delta_length=delta_length,
        cross_section='strip',
        straight_x_top=heater_arm,
        straight_x_bot=heater_arm,
    )

    mzi_ref = c.add_ref(_mzi)
    c.add_ports(mzi_ref.ports)

    return c

mi_mzi_tunable = mzi_unbalanced_2x2_tunable(delta_length=50)
mi_mzi_tunable.plot()
```



Learning outcome #4 – Notebook section 4. Exercises – A3

```
@gf.cell
def ring_allpass_mmi_lab(extra_length=0.0):
    c = gf.Component("allpass_ring_mmi")

    # Preparamos la sección transversal de 1.0 um para igualar los puertos del MMI
    xs_anillo = gf.cross_section.strip(width=1.0, layer='WG')

    # --- 1. Mantra: Add references ---
    # Instanciamos el MMI 2x2 de la librería del PDK
    mmi = c.add_ref(cells.mmi2x2())

    # Calculamos la distancia exacta entre el puerto superior izquierdo (o2)
    # y el superior derecho (o3). Esto garantiza que el anillo cierre perfecto.
    dist_x = mmi.ports['o3'].dx - mmi.ports['o2'].dx

    # Instanciamos la guía superior (el "techo" del racetrack)
    top_wg = c.add_ref(gf.components.straight(length=dist_x, cross_section=xs_anillo))

    # --- 2. Mantra: Move them ---
    # Alineamos la guía superior exactamente encima de los puertos del MMI (Eje X)
    top_wg.dmovex(mmi.ports['o2'].dx - top_wg.ports['o1'].dx)

    radio = 20.0
    altura = (radio * 2) + (extra_length / 2.0)
    top_wg.dmovey(mmi.ports['o2'].dy + altura - top_wg.ports['o1'].dy)

    # --- 3. Mantra: Connect them ---

    # Lado derecho: conectamos puerto Este con puerto Este
    gf.routing.route_single(
        c,
        port1=mmi.ports['o3'],
        port2=top_wg.ports['o2'],
        cross_section=xs_anillo,
        radius=radio
    )

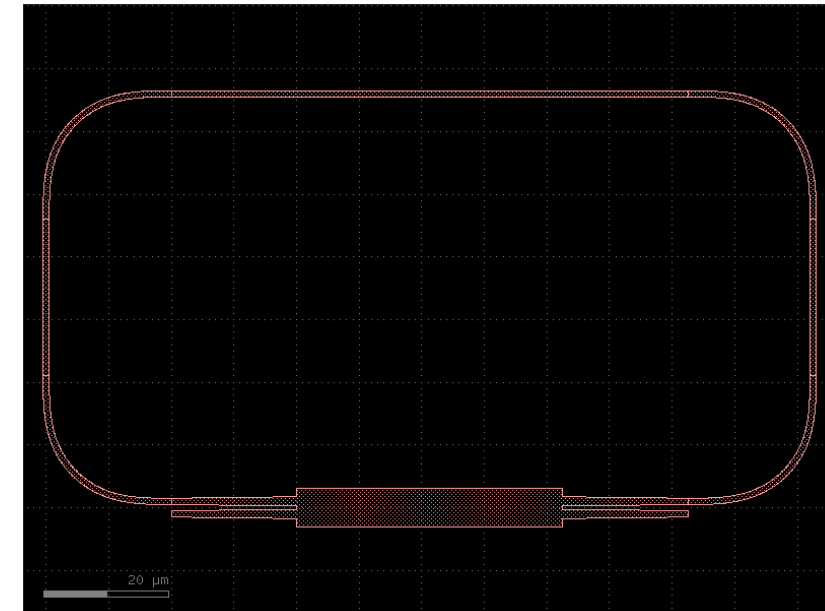
    # Lado izquierdo: conectamos puerto Oeste con puerto Oeste
    gf.routing.route_single(
        c,
        port1=top_wg.ports['o1'],
        port2=mmi.ports['o2'],
        cross_section=xs_anillo,
        radius=radio
    )

    # --- 4. Exponemos los puertos (Bus de entrada/salida) ---
    c.add_port("in", port=mmi.ports['o1'])
    c.add_port("out", port=mmi.ports['o4'])

    return c

# Probamos el componente generándole un perímetro extendido
mi_anillo = ring_allpass_mmi_lab(extra_length=50.0)

# Visualización en notebook y en KLayout
mi_anillo.plot()
mi_anillo.show()
```



Learning outcome #4 – Notebook section 4. Exercises – A4

```
PDK.activate()

# 1. Envoltorio para limpiar parámetros ópticos
def heater_arm(**kwargs):
    kwargs.pop('cross_section', None)
    return gf.components.straight_heater_metal(**kwargs)

@gf.cell
def ring_allpass_mmi_heater_lab(extra_length=0.0):
    c = gf.Component("allpass_ring_heater")

    xs_anillo = gf.cross_section.strip(width=1.0, layer='WG')

    # --- 1. Mantra: Add references ---
    mmi = c.add_ref(cells.mmi2x2())
    dist_x = mmi.ports['o3'].dx - mmi.ports['o2'].dx

    # Usamos función heater_arm pasándole solo la longitud
    heater = c.add_ref(heater_arm(length=dist_x))

    # --- 2. Mantra: Move them ---
    # Lo centramos en el eje X
    heater.dmovex(mmi.ports['o2'].dx - heater.ports['o1'].dx)

    radio = 20.0
    altura = (radio * 2) + (extra_length / 2.0)
    heater.dmovey(mmi.ports['o2'].dy + altura - heater.ports['o1'].dy)

    # --- 3. Mantra: Connect them ---
    gf.routing.route_single(
        c,
        port1=mmi.ports['o3'],
        port2=heater.ports['o2'],
        cross_section=xs_anillo,
        radius=radio
    )

    gf.routing.route_single(
        c,
        port1=heater.ports['o1'],
        port2=mmi.ports['o2'],
        cross_section=xs_anillo,
        radius=radio
    )

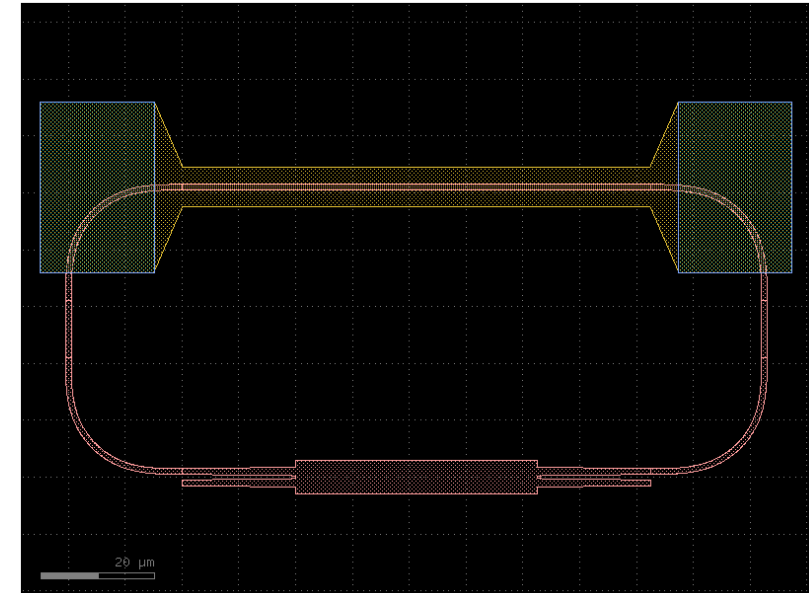
    c.add_port("in", port=mmi.ports['o1'])
    c.add_port("out", port=mmi.ports['o4'])

    # Cogemos el primer pad de la izquierda (l_e1) y el primero de la derecha (r_e1)
    c.add_port("e1", port=heater.ports['l_e1'])
    c.add_port("e2", port=heater.ports['r_e1'])

    return c

# Probamos el componente
ni_anillo_tunable = ring_allpass_mmi_heater_lab(extra_length=20.0)

# Visualización
ni_anillo_tunable.plot()
ni_anillo_tunable.show()
```



Learning outcome #4 – Notebook section 4. Exercises – A5

```
PDK.activate()

# Definimos la sección transversal optimizada del Lab 2 (ancho de 1.2 um)
xs_guia_ancha = gf.cross_section.strip(width=1.2, layer='WG', radius_min=10.0)

@gf.cell
def espiral_retraso_lab(length=1000.0, n_loops=6):
    # Creamos el componente contenedor
    c = gf.Component("espiral_retraso")

    # --- 1. Mantra: Add references ---
    espiral_base = cells.spiral(
        length=length,
        cross_section=xs_guia_ancha,
        spacing=6.0,
        n_loops=n_loops
    )

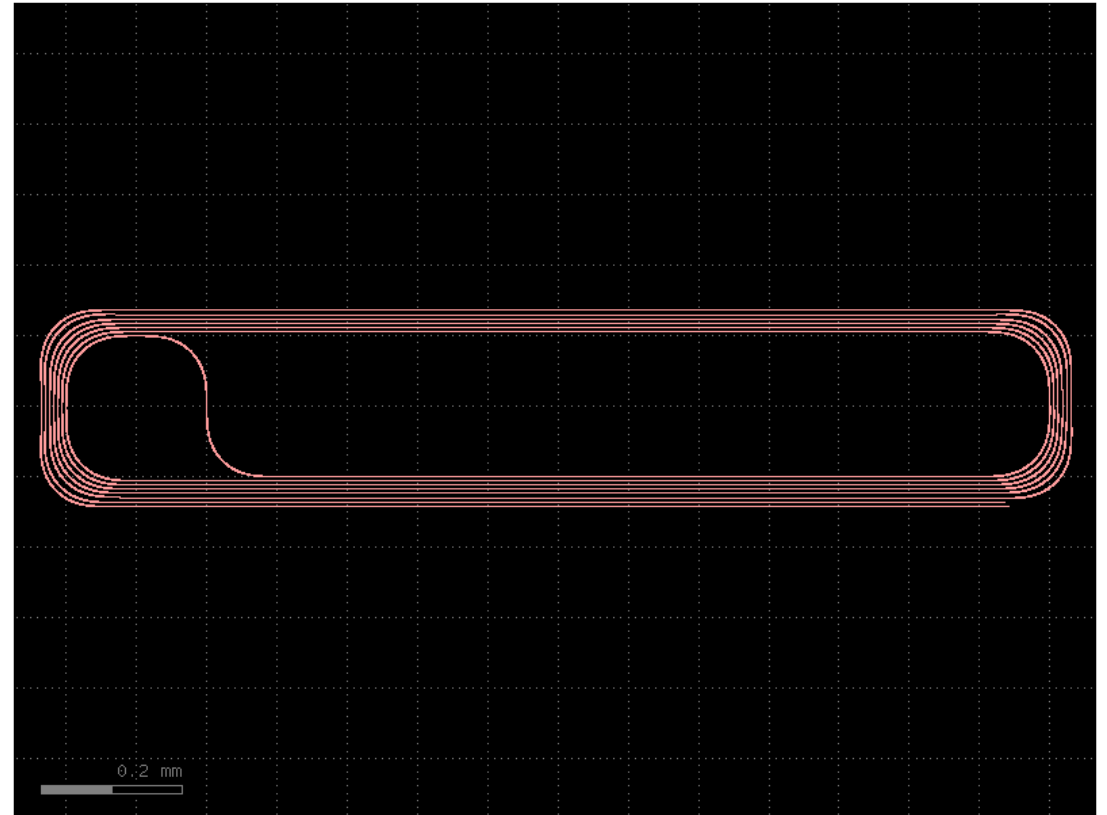
    espiral_inst = c.add_ref(espiral_base)

    # --- 2. Mantra: Connect them ---
    c.add_ports(espiral_inst.ports)

    return c

# Espiral de 1000 micras
mi_espiral_1 = espiral_retraso_lab(length=1000.0)
mi_espiral_1.plot()
mi_espiral_1.show()

# Comprobación de puertos por consola
print("Puertos de la espiral:", mi_espiral_1.ports)
```



Learning outcome #4 – Notebook section 4. Exercises – B1

```
import gdsfactory as gf
from upvfab.sin300.cband import PDK

# Inicialización
gf.clear_cache()
PDK.activate()

wgv = 0.5
dieW = 5000.0 # 5 mm de ancho
dieL = 10000.0 # 10 mm de largo
border = 125.0
layer_wg = "WG"
layer_box = "FLOORPLAN"

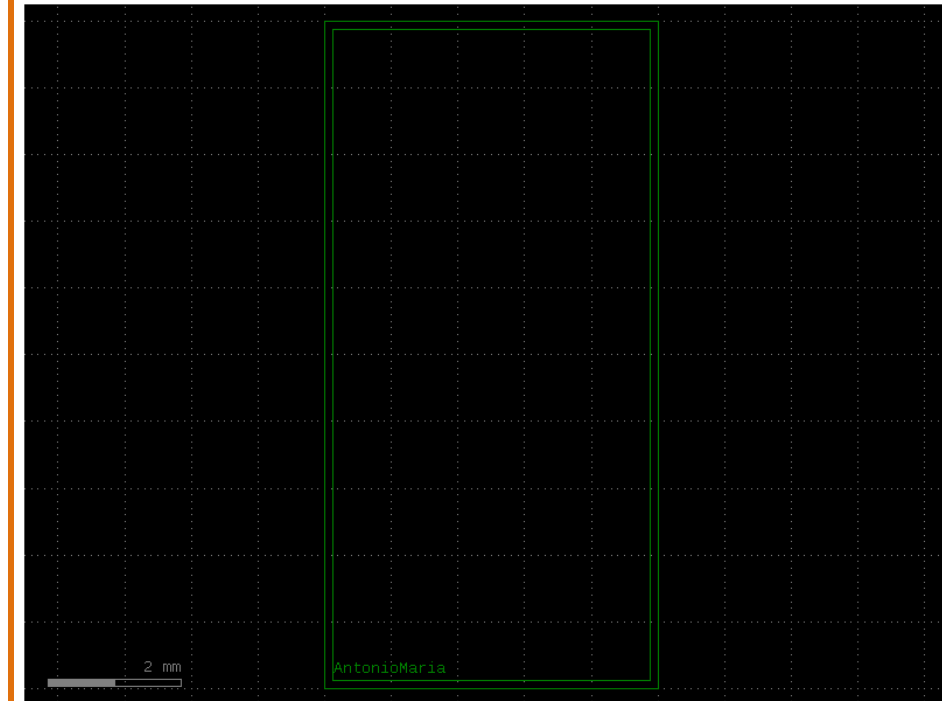
# Die specifications (Chip)
box = gf.Component("die_5x10_lab")

# Creamos las cajas exterior e interior
obox = box.add_ref(gf.components.rectangle(size=(dieW, dieL), layer=layer_box))
ibox = box.add_ref(gf.components.rectangle(size=(dieW - border*2, dieL - border*2), layer=layer_box)).dmovex(border).dmovey(border)

box = gf.boolean(A=obox, B=ibox, operation="A-B", layer=layer_box)

# Adding ports to a component (El punto de origen para la sierra de corte)
box.add_port(name="AntonioMaria", center=[border, border], width=1.0, orientation=0, layer=layer_box)

box.draw_ports()
box.plot()
box.show()
```



Learning outcome #4 – Notebook section 4. Exercises – B2

```
dieW = 5000.0
dieL = 10000.0
border = 125.0
io_pitch = 250.0
io_length = 250.0

xs_deep = gf.cross_section.strip(width=1.2, layer='WG', radius_min=10.0)

@gf.cell
def chip_io_arrays(chip_width=5000.0, chip_length=10000.0, margen=125.0, pitch=250.0, length=250.0):
    c = gf.Component("io_channels_array")

    n_ios = int((chip_length - 2 * margen) // pitch)

    # Punto de inicio para que queden perfectamente centradas en el chip
    y_start = (chip_length / 2.0) - ((n_ios - 1) * pitch / 2.0)

    # 2. BUCLE: Instanciamos y colocamos las guías (Mantra: Add & Move them)
    for i in range(n_ios):
        y_pos = y_start + i * pitch

        # --- Guías de la Izquierda ---
        wg_l = c.add_ref(gf.components.straight(length=length, cross_section=xs_deep))
        wg_l.dmovex(0).dmovex(y_pos)

        c.add_port(name=f'l_{i}', port=wg_l.ports['o2'])

        # --- Guías de la Derecha ---
        wg_r = c.add_ref(gf.components.straight(length=length, cross_section=xs_deep))

        wg_r.drotate(180).dmovex(chip_width).dmovex(y_pos)

        c.add_port(name=f'r_{i}', port=wg_r.ports['o2'])

    print(f"Array generado: {n_ios} guías a cada lado (Separación: {pitch} µm).")
    return c

io_arrays = chip_io_arrays(
    chip_width=dieW,
    chip_length=dieL,
    margen=border,
    pitch=io_pitch,
    length=io_length
)

io_arrays.plot()
io_arrays.show()
```



Learning outcome #4 – Notebook section 4. Exercises – B3

```
dieW = 5000.0
dieL = 10000.0
border = 125.0
io_pitch = 250.0
io_length = 250.0
layer_box = "FLOORPLAN"
xs_deep = gf.cross_section.strip(width=1.2, layer='WG', radius_min=10.0)

@gf.cell
def die_with_ios_lab():
    c = gf.Component("die_completo_ios")

    tmp = gf.Component()
    obox = tmp.add_ref(gf.components.rectangle(size=(dieW, dieL), layer=layer_box))
    ibox = tmp.add_ref(gf.components.rectangle(size=(dieW - border*2, dieL - border*2), layer=layer_box))
    ibox.dmovex(border).dmovey(border)

    marco = gf.boolean(A=obox, B=ibox, operation="A-B", layer=layer_box)
    c.add_ref(marco)

    # --- 2. Array de Canales I/O en los laterales ---
    n_ios = int((dieL - 2 * border) // io_pitch)
    y_start = (dieL / 2.0) - ((n_ios - 1) * io_pitch / 2.0) # Centrado en Y

    for i in range(n_ios):
        y_pos = y_start + i * io_pitch

        # Guía Izquierda (mirando hacia dentro)
        wg_l = c.add_ref(gf.components.straight(length=io_length, cross_section=xs_deep))
        wg_l.dmovex(0).dmovey(y_pos)
        c.add_port(name=f'io_l_{i}', port=wg_l.ports['o2'])

        # Guía Derecha (rotada 180° mirando hacia dentro)
        wg_r = c.add_ref(gf.components.straight(length=io_length, cross_section=xs_deep))
        wg_r.drotate(180).dmovex(dieW).dmovey(y_pos)
        c.add_port(name=f'io_r_{i}', port=wg_r.ports['o2'])

    # Añadimos el puerto de test en el origen
    c.add_port(name="AntonioMaria", center=[border, border], width=1.0, orientation=0, layer=layer_box)

    # Para ver los nombres io_l_X e io_r_X en el plot
    c.draw_ports()

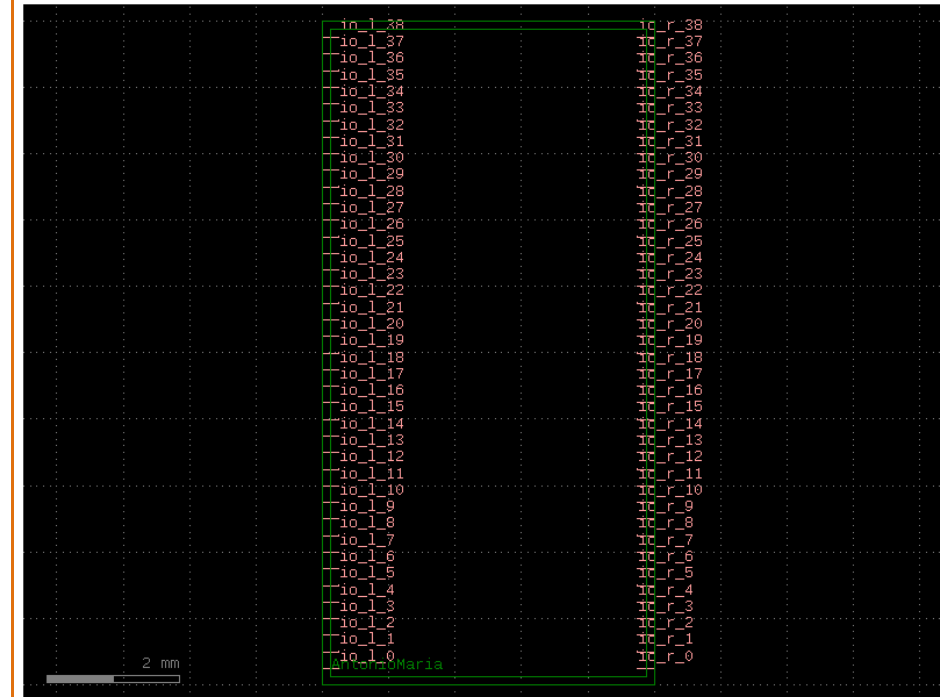
    return c
```

```
die_chip = die_with_ios_lab()

n_l = len([p for p in die_chip.ports if 'io_l' in p.name])

print(f"Tamaño final del chip: {die_chip.dysize:.0f} um x {die_chip.dysize:.0f} um")
print(f"Número de canales calculados por lateral: {n_l} guías")

# Visualización
die_chip.plot()
die_chip.show()
```



Learning outcome #4 – Notebook section 4. Exercises – C1

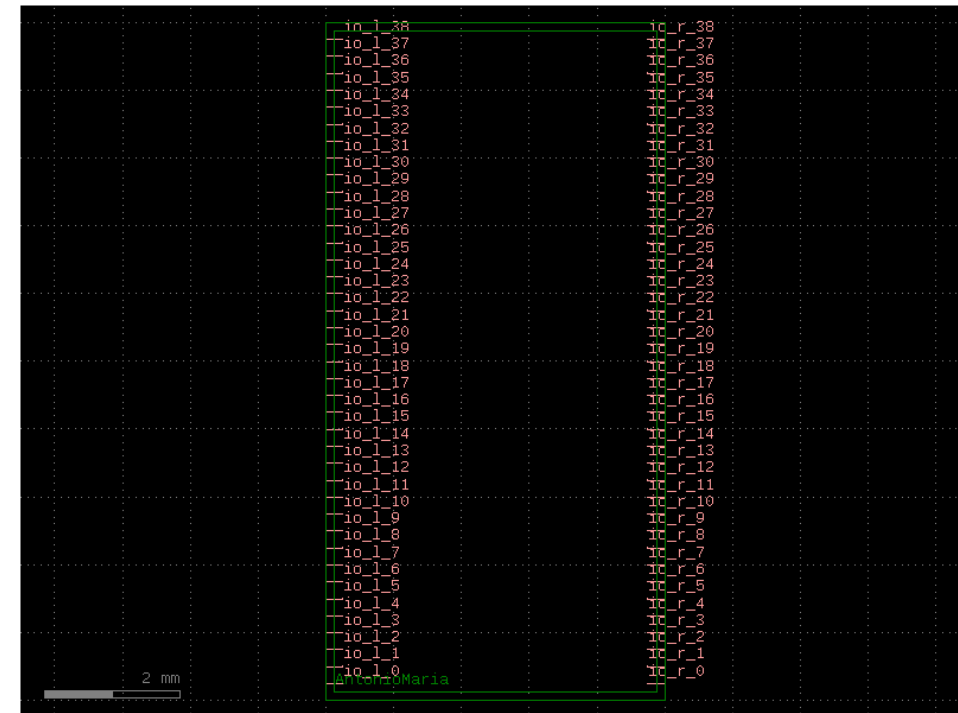
```
layout_final = gf.Component("chip_completo_host")

# --- 1. Mantra: Add references ---
die_ref = layout_final.add_ref(die_with_ios_lab())

layout_final.add_ports(die_ref.ports)

print(f"Componente Host creado: '{layout_final.name}'")

layout_final.plot()
layout_final.show()
```



Learning outcome #4 – Notebook section 4. Exercises – C2

```
xs_deep = gf.cross_section.strip(width=1.2, layer='WG', radius_min=10.0)

# --- 1. LÓGICA DE SELECCIÓN DE PUERTOS ---
# Calculamos de nuevo cuántos I/Os totales teníamos por lado
n_ios = int((diel - 2 * border) // io_pitch)

# Calculamos el índice central
mitad = n_ios // 2

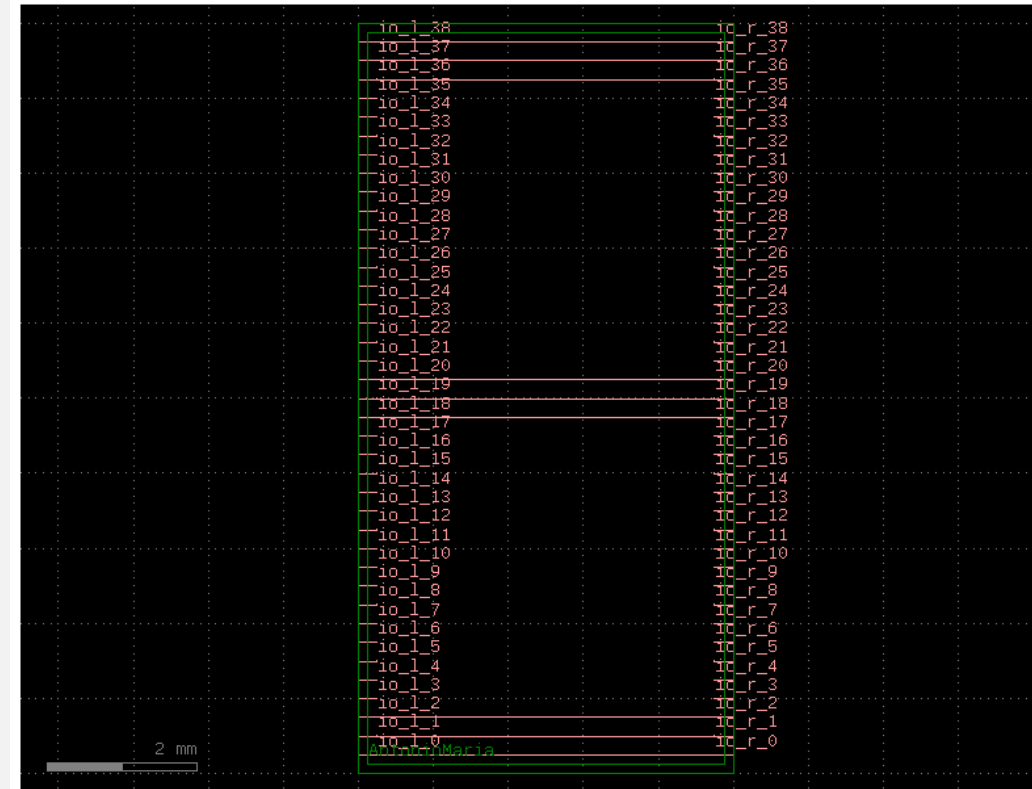
# Definimos los 3 sets de 3 guías (Bottom, Middle, Top)
indices_objetivo = [
    0, 1, 2,                # Set 1: Bottom (Los 3 primeros)
    mitad - 1, mitad, mitad + 1, # Set 2: Middle (Los 3 del centro)
    n_ios - 3, n_ios - 2, n_ios - 1 # Set 3: Top (Los 3 últimos)
]

print(f"Ruteando los puertos: {indices_objetivo}")

# --- 2. MANTRA: Connect them ---
# Iteramos sobre nuestra lista de puertos elegidos
for i in indices_objetivo:
    # Cogemos el puerto izquierdo y el derecho correspondientes
    puerto_izq = layout_final.ports[f'io_l_{i}']
    puerto_der = layout_final.ports[f'io_r_{i}']

    gf.routing.route_single(
        component=layout_final,
        port1=puerto_izq,
        port2=puerto_der,
        cross_section=xs_deep
    )

layout_final.plot()
layout_final.show()
```



Learning outcome #4 – Notebook section 4. Exercises – C3 Y C4

```
cx = dieW / 2.0

# Dos columnas alternas: una a la izquierda del centro, otra a la derecha
COL_IZQ = 1500.0 # columna "par"
COL_DER = 3000.0 # columna "impar" <-- desplazada a la derecha

def place_and_connect_lab(componente, io_idx, in_port='o1', out_port='o2', col_x=1500.0):
    """Posiciona el componente en la columna col_x y lo rutea a los I/O del die."""
    ref = layout_final.add_ref(componente)

    y_objetivo = layout_final.ports[f'io_l_{io_idx}'].dy

    # Alineación en X (columna indicada) e Y (centrado con el puerto de entrada)
    desplazamiento_x = col_x - ref.ports[in_port].dx
    desplazamiento_y = y_objetivo - ref.ports[in_port].dy
    ref.dmovex(desplazamiento_x).dmovex(desplazamiento_y)

    # Ruteo entrada (I/O izquierdo -> componente)
    gf.routing.route_single(
        layout_final,
        port1=layout_final.ports[f'io_l_{io_idx}'],
        port2=ref.ports[in_port],
        cross_section=xs_deep,
        allow_width_mismatch=True,
    )

    # Ruteo salida (componente -> I/O derecho)
    gf.routing.route_single(
        layout_final,
        port1=ref.ports[out_port],
        port2=layout_final.ports[f'io_r_{io_idx}'],
        cross_section=xs_deep,
        allow_width_mismatch=True,
    )

print("\nDistribuyendo en ZIG-ZAG para evitar solapes...")
```

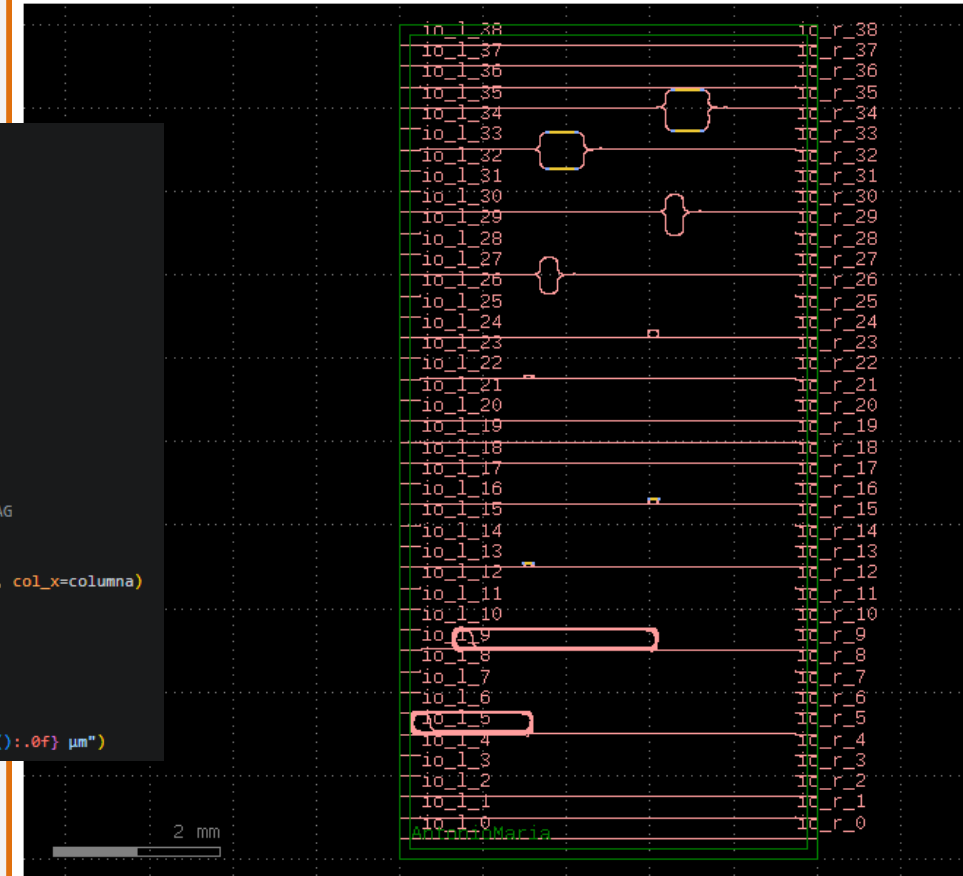
```
print("\nDistribuyendo en ZIG-ZAG para evitar solapes...")

# Lista de (componente, io_idx, in_port, out_port)
plan = [
    (espiral_retraso_lab(length=1000.0), 5, 'o1', 'o2'),
    (espiral_retraso_lab(length=2000.0), 9, 'o1', 'o2'),
    (ring_allpass_mmi_heater_lab(extra_length=0.0), 13, 'in', 'out'),
    (ring_allpass_mmi_heater_lab(extra_length=40.0), 16, 'in', 'out'),
    (ring_allpass_mmi_lab(extra_length=0.0), 22, 'in', 'out'),
    (ring_allpass_mmi_lab(extra_length=100.0), 24, 'in', 'out'),
    (mzi_balanced_2x2(delta_length=50.0), 27, 'o1', 'o3'),
    (mzi_balanced_2x2(delta_length=150.0), 30, 'o1', 'o3'),
    (mzi_unbalanced_2x2_tunable(delta_length=50.0), 33, 'o1', 'o3'),
    (mzi_unbalanced_2x2_tunable(delta_length=150.0), 35, 'o1', 'o3'),
]

# Alternamos columna: pares -> izquierda, impares -> derecha ==> ZIG-ZAG
for i, (comp, idx, p_in, p_out) in enumerate(plan):
    columna = COL_IZQ if i % 2 == 0 else COL_DER
    place_and_connect_lab(comp, io_idx=idx, in_port=p_in, out_port=p_out, col_x=columna)

print("\nMáscara final generada con zig-zag!")
layout_final.plot()
layout_final.show()

bbox = layout_final.dbbox()
print(f"Dimensiones de fabricación: {bbox.width():.0f} μm x {bbox.height():.0f} μm")
```





Thank you